



UNIVERSITY OF JOHANNESBURG

COLLEGE OF BUSINESS AND ECONOMICS

MASTERS IN FINANCIAL ENGINEERING

Block chain Assignment 1

Authors:

Siphosethu Limane

Student Numbers:

226214675

Submission Date: 04 August 2026

Assignment Report

The objective of this assignment was to implement the SHA-256 hashing algorithm using Python's `hashlib` library. The assignment also demonstrates the avalanche effect and implements a simple proof-of-work algorithm to illustrate how hashing is used in secure systems.

Question 1: SHA-256 Digests of Strings and Files

In this question, two functions were implemented. The first function computes the SHA-256 digest of a string, while the second computes the SHA-256 digest of a file. The file is read in small chunks before the digest is generated, making the program suitable for both small and large files.

The results show that every input produces a unique fixed-length hash value. Whether the input is a short string or a document, the output is always a 64-character hexadecimal digest. This digest acts as a digital fingerprint of the data. If the original string or file changes, a completely different digest will be produced, making it easy to detect any modifications.

Question 2: Avalanche Effect

The avalanche effect was demonstrated by changing only one character of the input. The original input was `financial`, while the modified input was `Financial`, where only the first letter was changed from lowercase to uppercase.

The resulting digests were completely different:

`financial`

`fed0e4820af42571c936e28300ccb88026f72e075232358cd1fae4e802fe902b`

`Financial`

`e97ba691189f51adb7edbd2c409603dd5e103d7642d7cfd5f4d2654935543492`

Although only one character changed, the hash values were entirely different. This demonstrates the avalanche effect, where even the smallest change in the input produces an unpredictable output. This property is useful in financial systems because any alteration to a transaction or record can be detected immediately.

Question 3: Proof-of-Work Toy

A simple proof-of-work algorithm was implemented by repeatedly combining a message with a nonce and computing its SHA-256 hash. The program continued searching until it found a hash beginning with a specified number of leading hexadecimal zeros.

The results obtained were:

n	Attempts	Time (s)	Digest Prefix
3	4 941	0.0268	0007ba941eb66a5a
4	51 119	0.2873	0000b711dff33b19

The results show that increasing the number of required leading zeros makes the problem much more difficult. When $n = 3$, the program found a solution after approximately 4,941 attempts. Increasing the difficulty to $n = 4$ required over 51,000 attempts and took much longer to complete. This demonstrates that proof-of-work becomes increasingly computationally expensive as the difficulty level increases.

Question 4: Reflection

Hashing is an important security tool used in financial systems to maintain the integrity of transaction records. A hash function converts any input, such as a financial transaction, into a fixed-length output called a hash. Financial ledgers use hashing to ensure that records remain accurate and trustworthy.

One important property of a secure hash function is the pre-image resistance. This means that given only a hash value, it should be extremely difficult to determine the original input that produced it. This protects financial systems because attackers cannot easily recreate transaction data from a hash. If finding the original input requires too much time and computing power, scammers are discouraged from attempting such attacks.

Another key property is collision resistance, which means it should be extremely difficult to find two different inputs that produce the same hash value. This is important because attackers should not be able to modify transaction details, such as payment amounts, while producing the same hash. Although hashing is useful for verifying data integrity, it does not provide confidentiality. Hashing is not encryption and cannot be reversed to recover the original information. It also does not hide sensitive information, such as account details, because anyone with the original input can generate the same hash. In conclusion, pre-image resistance and collision resistance are essential properties that make hashing reliable for financial ledgers. They help detect tampering, reduce the risk of fraud, and ensure that financial records remain accurate. However, hashing alone is not enough for complete security and must be combined with other security measures such as encryption, digital signatures, and access control.

1 GitHub Repository

The source code for this assignment is available on GitHub:

<https://github.com/Siphosethu19D/BlockChain-/tree/main/Assignment%201>